DEEP LEARNING

Complete Reference Notes

From a Single Neuron to Neural Networks that Learn

A beginner-friendly, story-style guide covering fundamentals, the perceptron, forward & backward propagation, activation functions, loss functions, and optimizers.

Prepared for future reference · Somnath Singh

How to Read These Notes

These notes are written as one continuous story. Each idea is a stepping stone to the next, so it is best to read them in order the first time. Later, you can jump to any part using the contents below. Every technical word is explained in plain language before it is used, and the diagrams and formulas are drawn exactly the way they appear in the original handwritten notes.

The big story in one line

We want a computer to **learn from examples**. To do that we build a network of tiny decision units (**neurons**), let it **guess** an answer (forward propagation), **measure how wrong** the guess is (loss function), and then **nudge the network** to be a little less wrong next time (backward propagation + optimizers). Activation functions are what let the network learn *complex*, non-straight-line patterns.

Table of Contents

PART 1

Foundations — What Deep Learning Really Is

Before touching any maths, let us understand where deep learning sits and why the whole world suddenly started talking about it.

1.1 Three circles: AI, Machine Learning, Deep Learning

Think of three circles sitting inside one another. The largest is **Artificial Intelligence (AI)** — the broad dream of making machines behave intelligently. Inside it is **Machine Learning (ML)** — machines that learn patterns from data instead of being programmed with fixed rules. Inside ML sits **Deep Learning (DL)**, our topic. Deep learning tries to **mimic the human brain** using **multi-layered neural networks** — many small units connected in layers, just like neurons in our head.

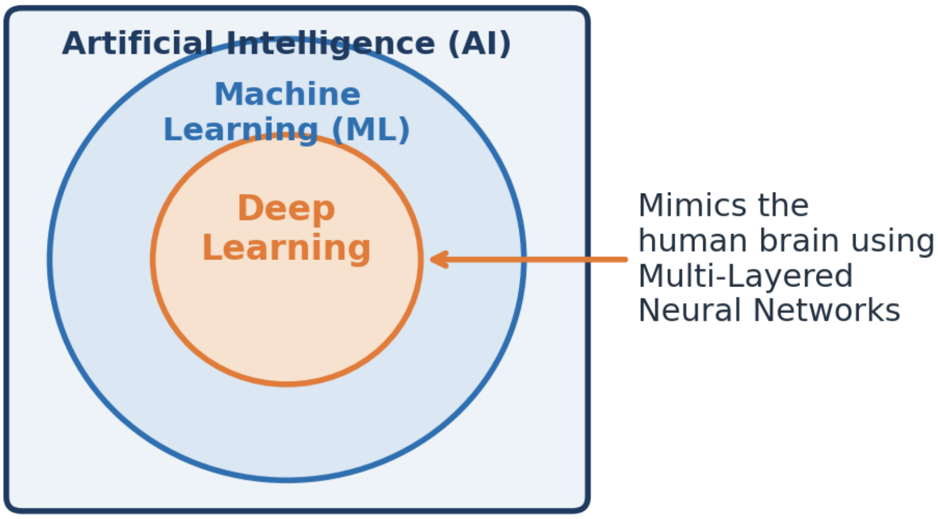


Figure 1.1 — Deep Learning is a specialised part of Machine Learning, which itself is a part of AI.

Key idea

The word **"deep"** simply means the network has **many layers** stacked one after another. More layers let the model understand more complex patterns.

1.2 The three main families of neural networks

Deep learning is not a single tool — it is a family. Depending on the kind of data you have, you pick a different type of network. The notes highlight three:

Network	Full form	Best used for	Popular examples
ANN	Artificial Neural Network	Classification & Regression on normal (tabular) data	Pass/Fail prediction
CNN	Convolutional Neural Network	Images and video frames (Computer Vision)	R-CNN, Mask R-CNN, YOLO v5/v6/v7
RNN	Recurrent Neural Network	Text and time-series (sequence data / NLP)	LSTM, GRU, Transformers, BERT

In simple words: use an **ANN** when your data is a plain table of numbers, a **CNN** when your input is an image, and an **RNN** (or its modern cousins like Transformers and BERT) when your input is a sequence such as a sentence or a series of readings over time. In this document we focus on the **ANN**, because understanding it teaches the ideas that power all the others.

1.3 Why did deep learning suddenly become popular?

Neural networks are an old idea, so why the recent excitement? Around 2005–2012 something changed. Social media platforms (Facebook, YouTube, WhatsApp, LinkedIn, Twitter) started generating **data exponentially** — images, text, documents and videos — the era of **Big Data**. Two things then came together:

- **Cheaper, faster hardware.** GPUs (mainly from NVIDIA) became affordable, and GPUs are brilliant at training neural networks quickly.
- **Huge amounts of data.** Deep learning models are hungry — the more data they get, the better they perform, unlike traditional ML which flattens out early.

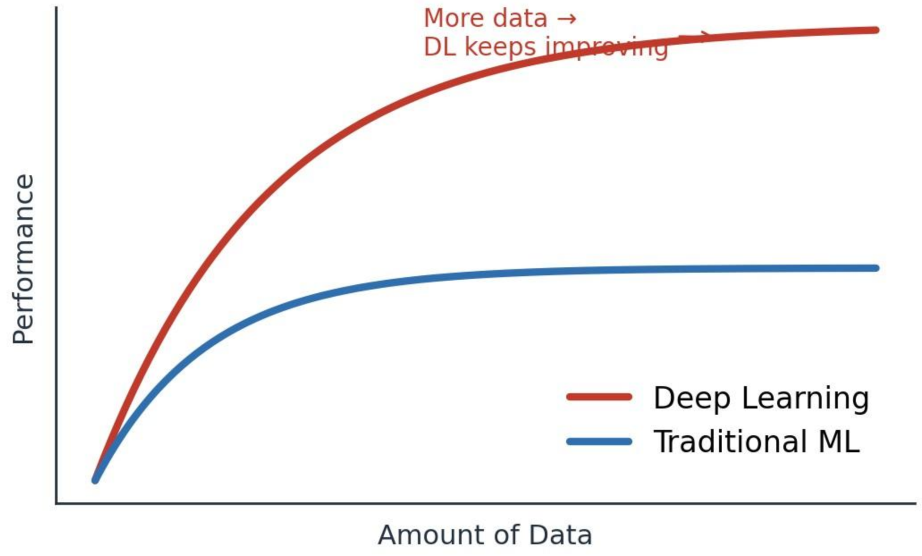


Figure 1.2 — With more data, deep learning keeps improving while traditional ML plateaus.

Because of this, deep learning is now used across **medical, e-commerce, retail, marketing** and many more domains. Open-source **frameworks** also lowered the barrier: **TensorFlow** (from Google) and **PyTorch** (from Facebook/Meta, popular in research) let anyone build end-to-end projects, and their large communities keep them growing.

Bridge to Part 2

We now know *what* deep learning is and *why* it matters. Next comes the natural question: what is the smallest piece a neural network is built from? Meet the **perceptron** — a single artificial neuron.

PART 2

The Perceptron — The Building Block

A neural network is just many tiny units working together. If you understand one unit, you understand the whole network. That unit is the perceptron.

2.1 What is a perceptron?

A **perceptron** is a single **artificial neuron** — the smallest neural network unit. It is also called a **single-layer neural network** and it works as a **binary classifier** (it answers questions with two possible outcomes, like Pass/Fail or Yes/No). It has four parts: an **input layer**, **weights**, a **bias**, and an **activation function**.

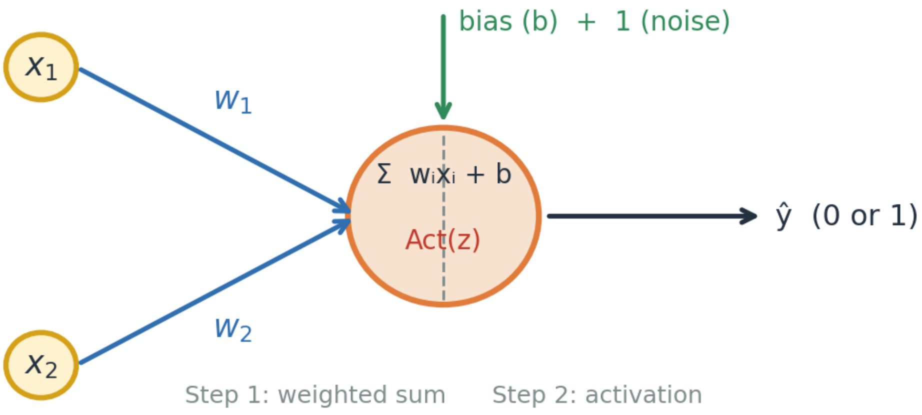


Figure 2.1 — A single neuron: inputs are multiplied by weights, a bias is added, then an activation decides the output.

2.2 What happens inside the neuron (two steps)

Step 1 — Weighted sum. Each input *x* is multiplied by its own **weight** *w* (a weight says how important that input is). We add all of these up and add a **bias** *b*. The result is called **z**:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

Step 2 — Activation. The number **z** can be anything. We pass it through an **activation function** that squashes it into a clean, useful output. The activation is what turns a plain sum into a decision.

2.3 Two early activation choices: Step vs Sigmoid

The very first idea was the **step function**: if z is above a threshold the neuron fires (outputs 1), otherwise 0. It is decisive but harsh — a tiny change in z can flip the answer completely. The **sigmoid** function is smoother: it turns z into a probability-like value between **0 and 1**, so we get a sense of *how* confident the neuron is, not just yes/no.

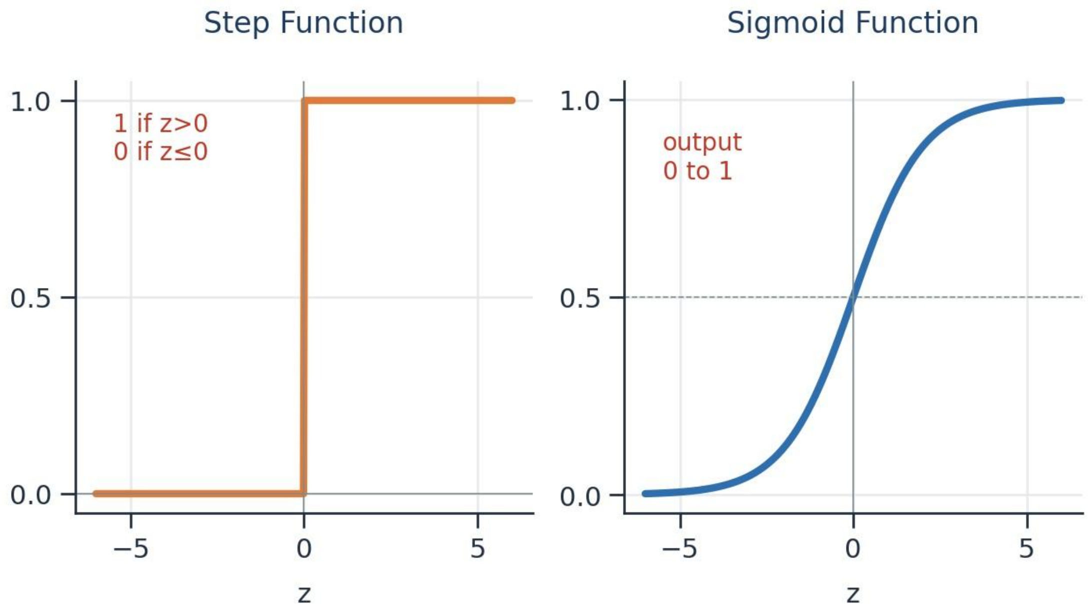


Figure 2.2 — The step function gives a hard 0/1; the sigmoid gives a smooth value between 0 and 1.

2.4 A perceptron is a straight-line (linear) classifier

Here is the important limitation. A single perceptron can only separate data with a **straight line**. If two groups of points can be split by a straight line, we call them **linearly separable**, and one perceptron is enough. But many real problems are **non-linearly separable** — no single straight line can divide them. For those, we must stack many neurons into layers, giving a **Multi-Layer Neural Network**.

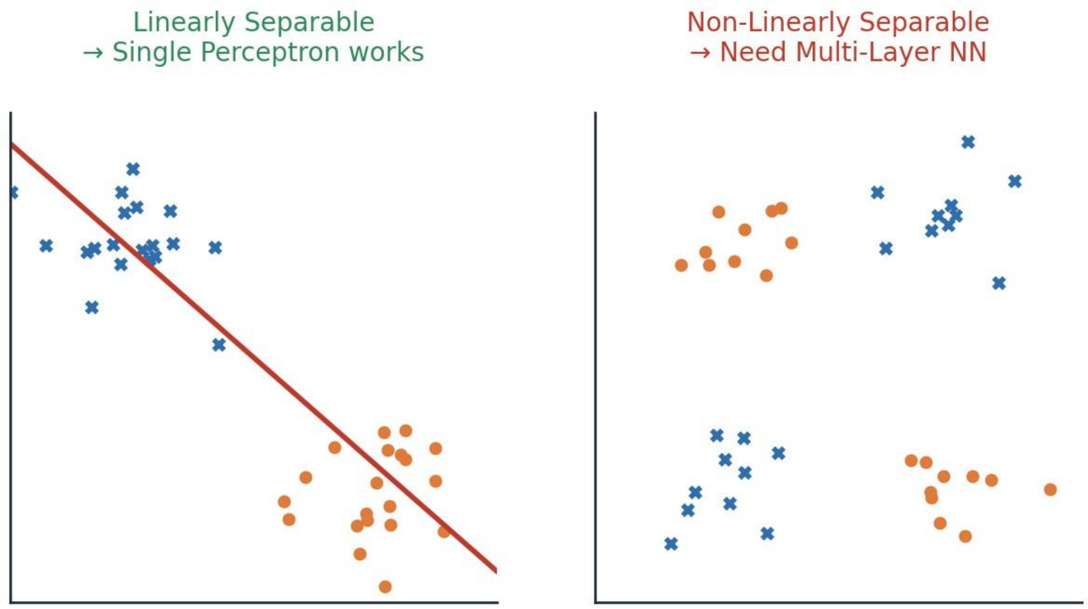


Figure 2.3 — Left: one straight line works. Right: no straight line works — we need multiple layers.

Bridge to Part 3

This single limitation is exactly why deep learning exists. By connecting many perceptrons across several layers, the network can bend and combine many straight lines into complex boundaries. That richer model is the **Multi-Layered Perceptron**, which we explore next.

PART 3

Multi-Layer Neural Networks (ANN)

Connect many neurons in layers and you get an Artificial Neural Network — the model that can learn almost any pattern. This is the heart of deep learning.

3.1 The five pillars of an ANN

A **Multi-Layered Perceptron** (the modern ANN, whose ideas were championed by **Geoffrey Hinton**) is built from five key concepts. Every remaining part of these notes is really just a deep dive into one of them:

- **Forward Propagation** — the network makes a guess.
- **Loss Function** — we measure how wrong the guess is.
- **Backward Propagation** — the error is sent back through the network.
- **Optimizers** — the weights are updated to reduce the error.
- **Activation Functions** — added at each neuron so the network can learn non-linear patterns.

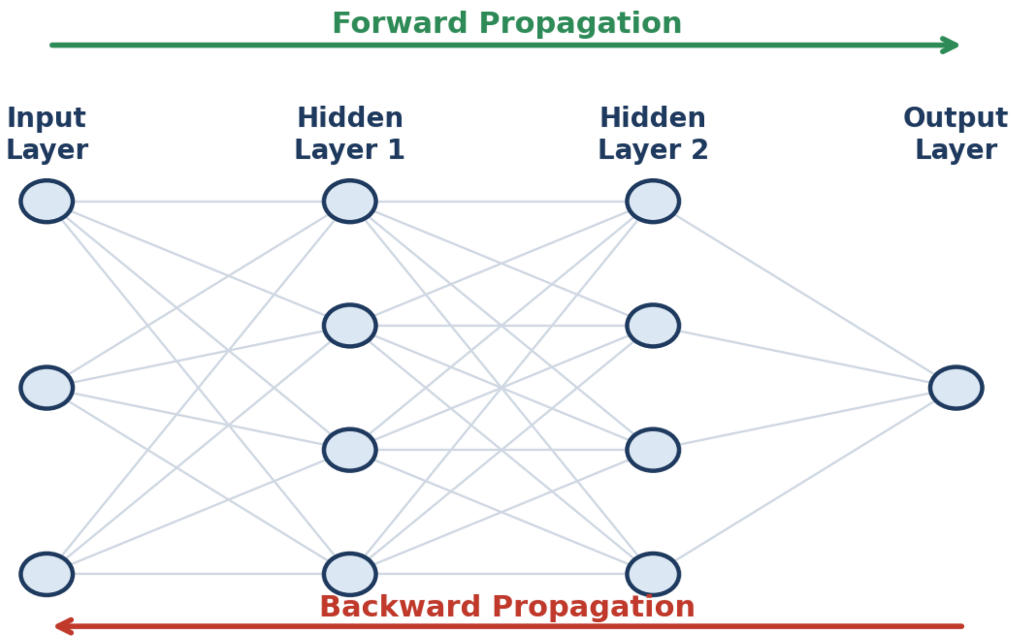


Figure 3.1 — Data flows left→right to make a prediction (forward), then the error flows right→left to learn (backward).

3.2 Forward propagation — making a guess

Forward propagation is simply Step 1 and Step 2 (from the perceptron) repeated layer by layer. Let us walk through the exact example from the notes: predicting whether a student will **Pass or Fail** using three inputs — **IQ**, **Study hours** and **Play hours**.

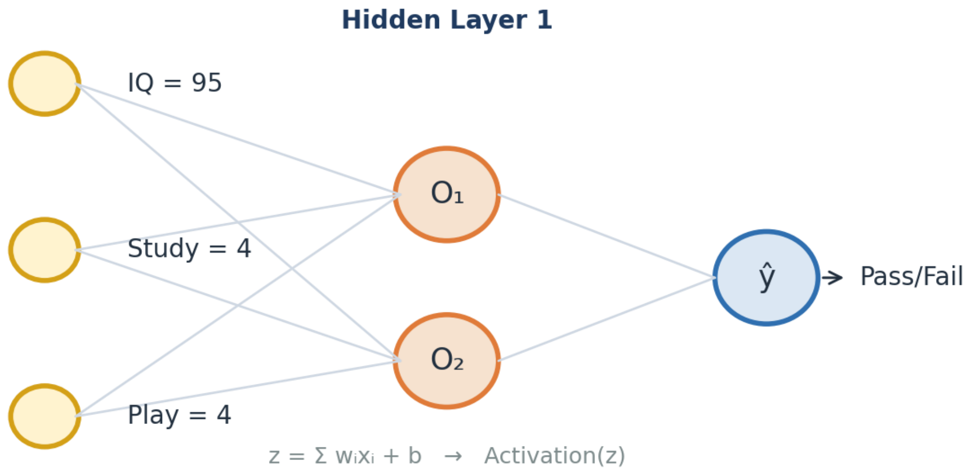


Figure 3.2 — Worked example: three inputs feed a hidden layer, which feeds the output neuron.

Take one student with IQ = 95, Study = 4, Play = 4, and weights **[0.01, 0.02, 0.03]** with bias **0.01**. In the first hidden neuron we compute z:

$$z = 95(0.01) + 4(0.02) + 4(0.03) + 1(0.01) = 1.151$$

Then we apply the sigmoid activation to squash z into a value between 0 and 1:

$$\sigma(1.151) = \frac{1}{1 + e^{-1.151}} = 0.759$$

So this neuron outputs **O■ = 0.759**. That output then becomes the input to the next layer, and we repeat the same two steps until we reach the output neuron, which produces the final prediction ■. That is the entire forward pass — multiply, add bias, activate, and pass forward.

3.3 Measuring the mistake, then learning from it

Once the network produces a prediction ■, we compare it with the true answer **y**. The gap between them is the **error**, and the **loss function** turns that error into a single number (we study loss functions in detail in Part 5). The whole goal of learning is to make this number as small as possible.

To shrink the error, the network adjusts its **weights**. But in which direction, and by how much? This is answered by **backward propagation** together with an **optimizer**, using one central formula — the **weight-update rule**:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

In words: the **new weight** equals the **old weight** minus the **learning rate** (η, a small number that controls step size) multiplied by the **gradient** (∂L/∂W, the slope that tells us how the loss changes when this weight changes). The bias is updated with the very same idea:

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$$

3.4 Gradient descent — walking downhill to the lowest error

Picture the loss as a valley. High up the slopes the error is large; at the very bottom — the **global minima** — the error is smallest. **Gradient descent** is the strategy of taking small steps downhill, again and again, until we reach the bottom. The **slope (gradient)** tells us which way is downhill, and the **learning rate** decides how big each step is.

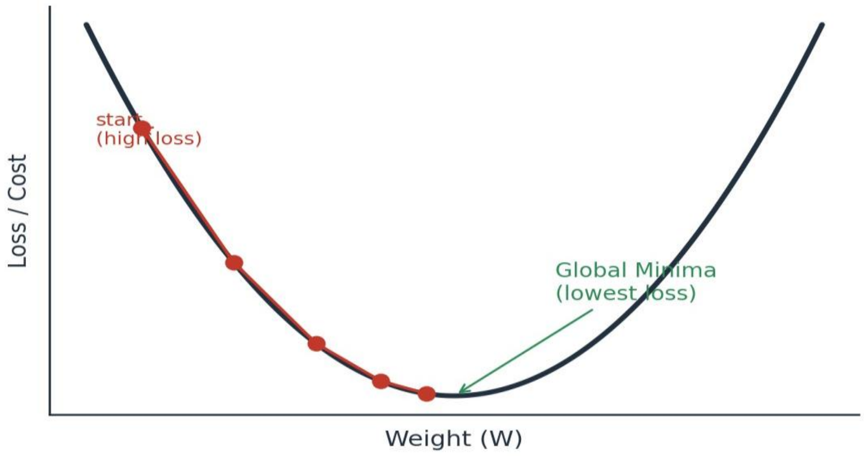


Figure 3.3 — Gradient descent takes repeated downhill steps toward the point of lowest loss (global minima).

Why the learning rate matters

If the learning rate is **too large**, we may leap over the bottom and never settle (it may never converge). If it is **too small**, learning is painfully slow. A common safe starting value is a small number like **0.001**. When the weight reaches the global minima, the update becomes almost zero, so W_new ≈ W_old and learning naturally stops.

3.5 The chain rule — how deep layers get their share of the blame

There is one puzzle left. The final error appears at the output, but a weight sitting deep in the network also helped cause it. How do we measure a hidden weight's effect on the final loss? The answer is the **chain rule of derivatives**. It links the loss to a deep weight by multiplying the small effects along the path from that weight all the way to the output:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial O_2} \cdot \frac{\partial O_2}{\partial O_1} \cdot \frac{\partial O_1}{\partial w_1}$$

Figure 3.4 — The chain rule multiplies the step-by-step effects to find how a deep weight influences the loss.

This is precisely what “backpropagation” means: the error is carried **backward**, and at each layer the chain rule hands every weight its fair share of responsibility, so the optimizer knows exactly how to adjust it. Keep this multiplication of small numbers in mind — it returns in Part 4 as the famous **vanishing gradient problem**.

PART 4

Activation Functions

Activation functions are the ingredient that lets a network learn curved, complex patterns instead of only straight lines. Here we meet each one, with its strengths and weaknesses.

4.1 Why do we even need them?

Without an activation function, every neuron would just do a weighted sum — and stacking many sums still gives only a straight-line relationship. The **activation function adds the "bend"**, letting the network model the non-linear patterns we saw in Part 2. It also controls the range of a neuron's output. Let us look at the whole family, in the order they were developed.

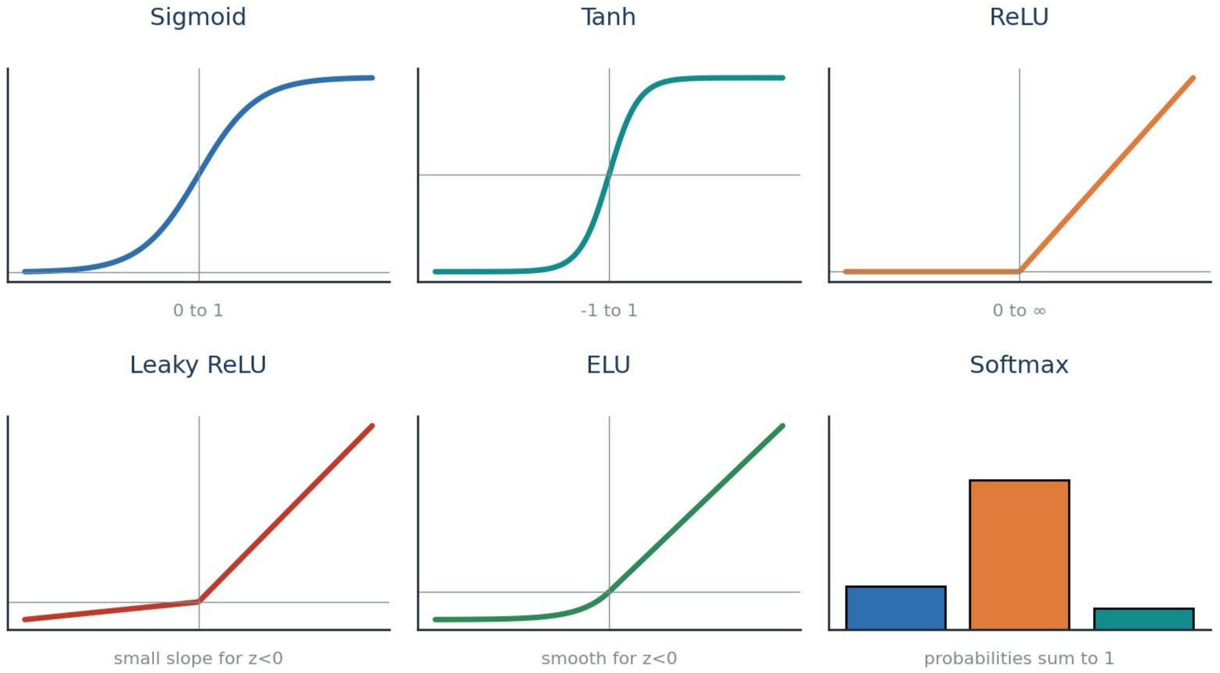


Figure 4.1 — The activation-function family at a glance.

4.2 Sigmoid (output range 0 to 1)

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid squashes any number into the range **0 to 1**, which makes it perfect for **binary classification** output, where we want a probability. Its derivative, however, is small (only 0 to 0.25), and that becomes a problem in deep networks.

✓ Advantages

- Great for binary classification
- Gives a clear, probability-like output near 0 or 1

✗ Disadvantages

- Prone to the vanishing gradient problem
- Output is not zero-centred → weight updates are less efficient
- Maths (exponentials) is relatively slow

4.3 Tanh (output range −1 to 1)

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Tanh looks like the sigmoid but is **zero-centred** (its output spans −1 to 1). Being zero-centred makes weight updates more efficient, which is why tanh is usually preferred over sigmoid in hidden layers.

✓ Advantages	✗ Disadvantages
• Zero-centred → more efficient weight updates • Stronger gradients than sigmoid	• Still prone to the vanishing gradient problem • Still computationally heavier (uses exponentials)

4.4 ReLU — Rectified Linear Unit

$$f(z) = \max(0, z)$$

ReLU is beautifully simple: if the input is positive, keep it; if negative, output 0. Because it is just a comparison, it is **extremely fast** and largely **solves the vanishing gradient problem** for positive values (its derivative is a clean 1). This made ReLU the default choice for hidden layers.

But it has a catch. If a neuron's input is always negative, ReLU always outputs 0, its derivative is 0, and the weight-update formula leaves the weight unchanged (W_new ≈ W_old). That neuron effectively **dies** and stops learning — the **Dead Neuron / Dying ReLU problem**.

✓ Advantages	✗ Disadvantages
• Solves vanishing gradient (positive side) • Very fast — just max(0, z) • Faster than sigmoid or tanh	• Dead Neuron problem (dies for negative inputs) • Output is not zero-centred

4.5 Leaky ReLU and Parametric ReLU

$$f(z) = \max(0.01z,\ z)$$

To cure the dead neuron, **Leaky ReLU** lets a tiny slope through for negative inputs (e.g. 0.01·z) instead of flat zero — so the neuron always has a small gradient and never fully dies. **Parametric ReLU** is the same idea, but the small slope (α, a hyperparameter like 0.01, 0.02, 0.03) is **learned** rather than fixed.

✓ Advantages	✗ Disadvantages
• Keeps all the benefits of ReLU • Removes the Dead ReLU problem (neurons stay alive)	• Still not perfectly zero-centred

4.6 ELU — Exponential Linear Unit

$$f(z) = z \quad (z > 0)\ ; \quad \alpha(e^{z} - 1) \quad (z \leq 0)$$

ELU keeps positive values as-is but uses a smooth exponential curve for negative values. This makes it **zero-centred** and completely avoids dead neurons — it was designed specifically to fix ReLU's problems.

✓ Advantages	✗ Disadvantages
• No Dead ReLU issues • Zero-centred output	• Slightly more computationally intensive (uses exponentials)

4.7 Softmax — for multi-class classification

$$\mathrm{softmax}(z_i) = \frac{e^{z_i}}{\sum_{k=1}^{K} e^{z_k}}$$

Everything so far handled a single output. But what if we must choose among **many** classes — say Cat, Dog, Monkey, Horse? **Softmax** sits at the output layer and converts the scores into **probabilities that add up to 1**. The class with the highest probability is the network's answer. In short: use **sigmoid** for binary output, and **softmax** for multi-class output.

4.8 The vanishing gradient problem — why activation choice matters

Remember the chain rule from Part 3: to update a deep weight we **multiply** many derivatives together. With sigmoid, each derivative is at most 0.25. Multiply several small numbers (0.25 × 0.25 × 0.25 …) and the result becomes almost **zero**. The gradient has **vanished**.

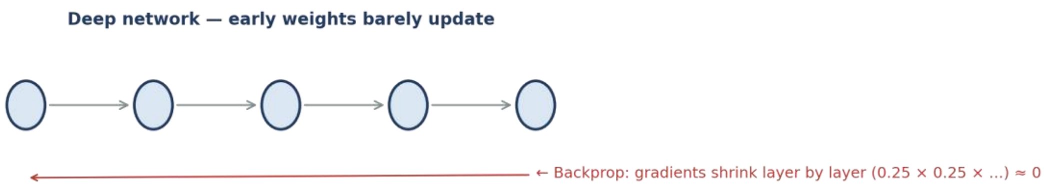


Figure 4.2 — As the error travels back through many layers, tiny derivatives multiply toward zero.

When the gradient is nearly zero, the weight-update formula gives W_new ≈ W_old — the early layers barely learn. This is the **vanishing gradient problem**, and it is the single biggest reason we moved from sigmoid/tanh to **ReLU and its variants** in hidden layers. Understanding this connects Parts 3 and 4 into one story: the maths of learning directly decides which activation function we should use.

Quick decision guide

Hidden layers: start with **ReLU**; switch to **Leaky ReLU / ELU** if neurons are dying.
Output — binary: use **Sigmoid**. **Output — multi-class:** use **Softmax**.
Avoid sigmoid/tanh deep inside big networks because of vanishing gradients.

4.9 Activation functions — side-by-side summary

Function	Output range	Main strength	Main weakness
Sigmoid	0 to 1	Binary output / probability	Vanishing gradient, not zero-centred
Tanh	-1 to 1	Zero-centred	Still vanishing gradient
ReLU	0 to ∞	Fast, fixes vanishing (positive)	Dead neuron problem
Leaky ReLU	small neg to ∞	No dead neurons	Not fully zero-centred
ELU	smooth neg to ∞	Zero-centred, no dead neurons	Slightly slower
Softmax	0 to 1 (sums to 1)	Multi-class output	Output layer only

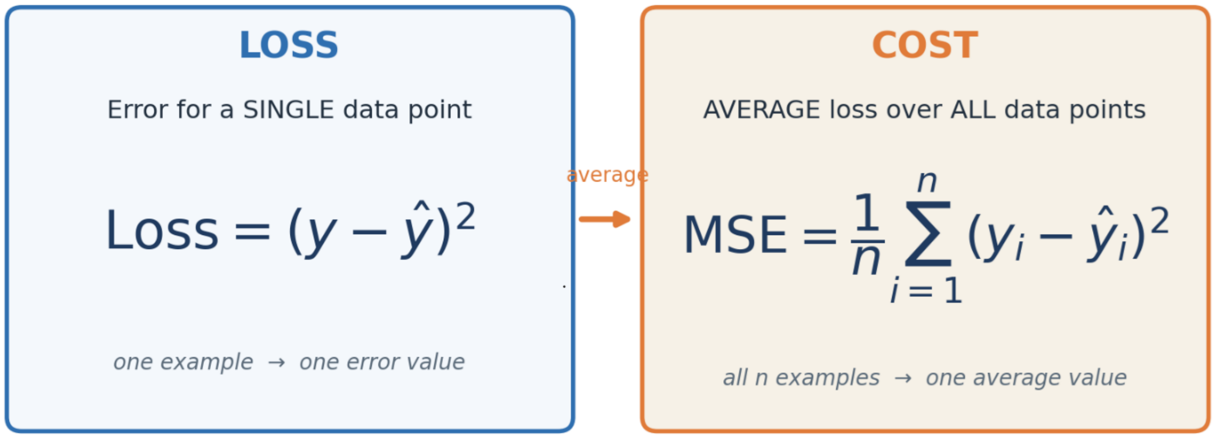
PART 5

Loss & Cost Functions

We keep saying “measure how wrong the network is.” This is the job of the loss function. Choosing the right one depends on whether you are predicting numbers or categories.

5.1 Loss vs Cost — a small but important difference

The two words are often mixed up, so let us be precise. **Loss** is the error for a **single** data point. **Cost** is the **average loss over all** data points. During forward propagation we get a loss for one example; the cost tells us how the whole dataset is doing.



Left: loss for one point. Right: cost = average loss over n points.

Loss functions split into two groups depending on the task: **Regression** (predicting a number, like price) and **Classification** (predicting a category, like Pass/Fail).

5.2 Regression losses

1) Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

MSE squares the error, so big mistakes are punished heavily. Because it is a smooth quadratic curve (a bowl shape), it is easy to do gradient descent on.

✓ Advantages
- Differentiable everywhere (smooth)
- Has one clear global minimum
- Converges faster

✗ Disadvantages
- Not robust to outliers — squaring makes a single bad point dominate the error

2) Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n} | y_i - \hat{y}_i |$$

MAE takes the absolute value instead of squaring, so a single outlier does not blow up the error.

✓ Advantages	✗ Disadvantages
• Robust to outliers	• Convergence usually takes more time (needs sub-gradients at the sharp point)

3) Huber Loss — the best of both

$$L_\delta = \tfrac{1}{2}(y - \hat{y})^2 \;\text{ if } |y - \hat{y}| \le \delta \;; \quad \delta|y - \hat{y}| - \tfrac{1}{2}\delta^2 \;\text{ else}$$

Huber loss is clever: for **small** errors it behaves like **MSE** (smooth), and for **large** errors it behaves like **MAE** (robust to outliers). The switch-over point is a **hyperparameter** δ **(delta)**. So it gives us MSE-style smoothness where errors are small, and MAE-style robustness where errors are large.

4) Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}$$

RMSE is simply the square root of MSE. Taking the root brings the error back to the same unit as the original target, which makes it easier to interpret in real-world terms.

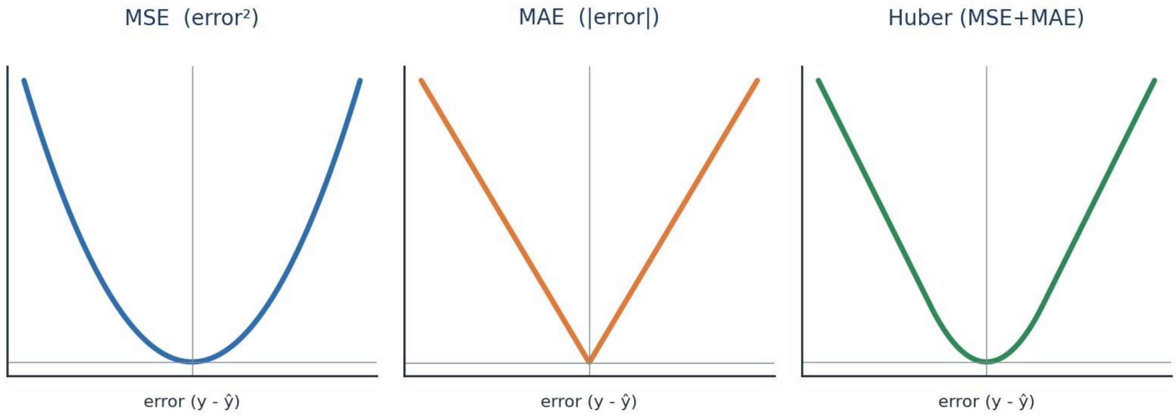


Figure 5.1 — Shapes of the three main regression losses. Notice how Huber blends the MSE bowl with the MAE V.

5.3 Classification losses — Cross-Entropy

For classification we do not use squared error; we use **Cross-Entropy**, which measures how far the predicted probability is from the true label. It comes in three flavours depending on the number of classes:

Loss	Use it when...	Label format
Binary Cross-Entropy	There are exactly 2 classes (yes/no)	0 or 1
Categorical Cross-Entropy	There are many classes	One-hot (e.g. [0,1,0])
Sparse Categorical Cross-Entropy	Many classes, but labels are integers	Integer (e.g. 2)

Binary Cross-Entropy — the classic two-class loss:

$$\text{Loss} = -\,y\log(\hat{y}) - (1-y)\log(1-\hat{y})$$

Reading it simply: if the true label **y = 1**, the loss is −log(■); if **y = 0**, the loss is −log(1−■). Either way, the closer the prediction is to the truth, the smaller the loss.

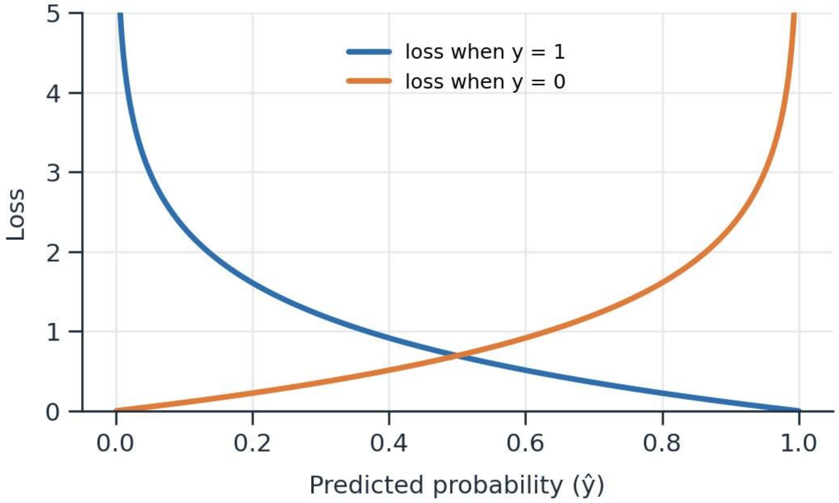


Figure 5.2 — Cross-entropy grows sharply when the prediction is confidently wrong.

Categorical Cross-Entropy — the multi-class version (used with softmax):

$$L(x_i, y_i) = -\sum_{j=1}^{C} y_{ij}\ln(\hat{y}_{ij})$$

Bridge to Part 6

We can now build a network, make a prediction, and measure the error with the right loss. The final question is the most practical one: **how exactly do we update the weights** to bring that loss down efficiently? That is the job of **optimizers**.

PART 6

Optimizers — How the Network Actually Learns

An optimizer is the exact recipe for updating weights so the loss goes down. Each optimizer here fixes a weakness of the one before it — so read them as an evolution.

6.1 The six optimizers, and how they build on each other

The notes list six optimizers. Rather than memorising them separately, notice the storyline: each new optimizer solves a specific problem left by the previous one.

#	Optimizer	The problem it solves
1	Gradient Descent	The basic downhill rule (but very resource-heavy)
2	Stochastic GD (SGD)	Fixes the resource problem (but adds noise)
3	Mini-Batch SGD	Balances speed and noise
4	SGD with Momentum	Smooths out the noise for faster convergence
5	Adagrad / RMSProp	Adapts the learning rate automatically
6	Adam	Combines Momentum + adaptive learning rate

6.2 Gradient Descent (the starting point)

First, a vocabulary reminder. One **epoch** = one full pass through the entire dataset. Plain **Gradient Descent** looks at **all** the data points, computes the average gradient, and then updates the weights **once** per epoch using the update rule from Part 3.

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

✓ Advantages

• Convergence is stable and reliable

✗ Disadvantages

• Extremely resource-intensive — needs huge RAM/GPU to hold all data at once (imagine 1 million points)

6.3 Stochastic Gradient Descent (SGD)

SGD fixes the memory problem by going to the opposite extreme: update the weights after **every single data point**. With 1,000 points, that is 1,000 updates per epoch. This solves the resource issue, but because each step is based on just one point, the path to the bottom is **noisy** and zig-zags a lot.

✓ Advantages

• Solves the resource / memory problem

✗ Disadvantages

• Introduces a lot of noise

• Higher time complexity

• Convergence can take longer

6.4 Mini-Batch SGD (the practical middle ground)

Why choose between "all data" and "one point" when we can do something in between? **Mini-Batch SGD** updates the weights after every small **batch** (for example, 1,000 points at a time). This is the version used in practice — it gives good speed, sensible memory use, and much less noise than pure SGD.

✓ **Advantages**	✗ **Disadvantages**
• Convergence speed increases • Much less noise than SGD • Efficient use of RAM/memory	• Some noise still exists

6.5 SGD with Momentum (smoothing the path)

Mini-batch still wobbles. **Momentum** smooths the journey by remembering the direction of previous steps — like a ball rolling downhill that keeps its momentum instead of reacting to every little bump. It does this using an **Exponential Weighted Average** of past gradients (the same smoothing idea used in time-series methods like ARIMA):

$$V_t = \beta\, V_{t-1} + (1-\beta)\, a_t$$

Here β **(beta)**, typically around 0.95, decides how much of the past to remember. A higher β means smoother movement. The result is **less noise and faster convergence**.

✓ **Advantages**	✗ **Disadvantages**
• Reduces the noise • Quicker convergence	• Introduces an extra hyperparameter (β) to tune

6.6 Adagrad and RMSProp (adaptive learning rates)

So far the learning rate η was **fixed**. But intuitively we want **big** steps early on and **small**, careful steps as we approach the bottom. **Adagrad** (Adaptive Gradient Descent) makes the learning rate **dynamic**: it shrinks the learning rate over time based on how much a weight has already been updated.

$$W_t = W_{t-1} - \frac{\eta}{\sqrt{\alpha_t + \varepsilon}} \cdot \frac{\partial L}{\partial W_{t-1}}$$

The term α simply adds up the squared gradients so far, and ε is a tiny number that prevents division t by zero:

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{\partial L}{\partial W_t}\right)^2$$

✓ **Advantages**	✗ **Disadvantages**
• Learning rate adapts automatically — no manual tuning of η over time	• The adjusted learning rate can shrink so much it becomes almost 0, and learning stops too early

RMSProp patches exactly this weakness by using a moving average of squared gradients (instead of summing them forever), so the learning rate does not collapse to zero.

6.7 Adam — the modern default

Finally, **Adam** combines the two best ideas we just met: the **momentum** smoothing from Section 6.5 and the **adaptive learning rate** from Adagrad/RMSProp. Because it inherits the strengths of both, Adam converges quickly and reliably, which is why it is the **default optimizer** for most deep learning projects today.

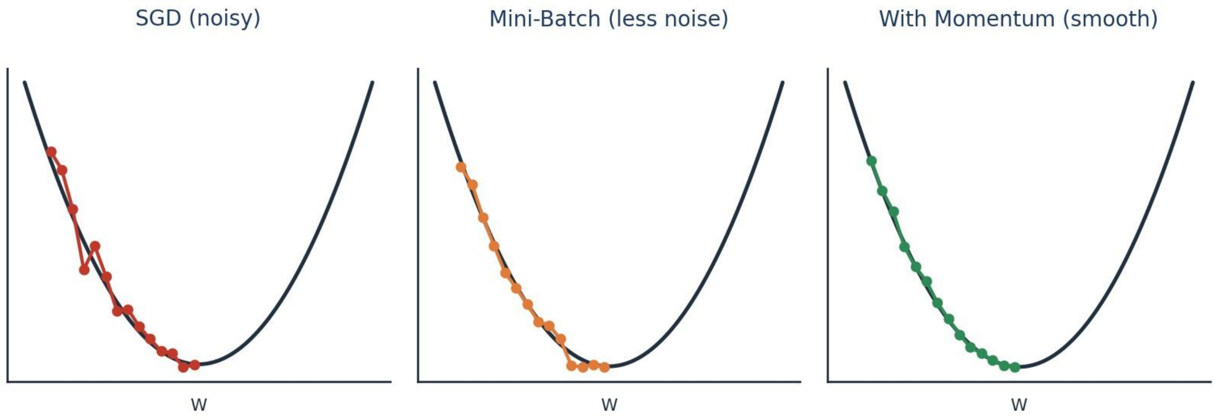


Figure 6.1 — From noisy SGD to smooth momentum: each optimizer takes a cleaner path to the minimum.

The Whole Picture, Connected

You have now travelled the entire journey. Let us tie every part back into the single story we started with.

1 · Foundations

Deep learning is a brain-inspired part of ML that thrives on big data and fast GPUs. Different data needs different networks — ANN for tables, CNN for images, RNN for sequences.

2 · The Perceptron

The smallest unit is a neuron: it takes a weighted sum of inputs, adds a bias, and applies an activation. Alone it can only draw a straight line, so we need many of them.

3 · Multi-Layer Networks

Stacked neurons make an ANN. It guesses via forward propagation, and learns via backpropagation + gradient descent, using the chain rule to share the error among all weights.

4 · Activation Functions

These add the non-linear 'bend'. Sigmoid/Tanh cause vanishing gradients, so ReLU and its variants power hidden layers, while Softmax handles multi-class outputs.

5 · Loss Functions

They score the mistake: MSE/MAE/Huber/RMSE for regression, and Cross-Entropy for classification. This score is exactly what learning tries to minimise.

6 · Optimizers

They perform the actual weight updates. From Gradient Descent to SGD, Mini-Batch, Momentum, Adagrad/RMSProp and finally Adam — each one fixes the flaw before it.

The one-paragraph summary

A neural network **guesses** (forward propagation with activation functions), **checks its mistake** (loss/cost function), and **improves** (backpropagation with the chain rule, driven by an optimizer that follows the gradient downhill). Repeat this over many epochs and the network slowly reaches the global minima — the point where it makes the fewest mistakes. That, in essence, is how deep learning learns.

— End of Notes —

The Exploding Gradient Problem & Weight Initialization

In Part 4 we met the vanishing gradient, where gradients shrink to almost nothing. Now we meet its opposite twin — when gradients grow far too large — and the simple fix that prevents both: initializing weights well.

7.1 Recalling how a weight learns

Everything here builds on the update rule from Part 3. During **backpropagation**, every weight is nudged using the same formula:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W_{old}}$$

The size of that nudge depends entirely on the **gradient** (∂L/∂W). If the gradient is a healthy, moderate number, the weight learns smoothly. But two things can go wrong — the gradient can become far too **small** or far too **large**:

$W_{new} \lll W_{old}$	$W_{new} \ggg W_{old}$

Left (Vanishing): the new weight is almost the same as the old one — the network stops learning (seen in Part 4). **Right (Exploding):** the new weight is *wildly* larger than the old one — the network becomes unstable. This second case is the **Exploding Gradient Problem**.

7.2 The network we will trace

Let us use the same simple chain from the notes: one input flows through three neurons, each with its own weight (W_1, W_2, W_3) and bias (b_1, b_2, b_3), until it reaches the prediction ŷ and the loss.

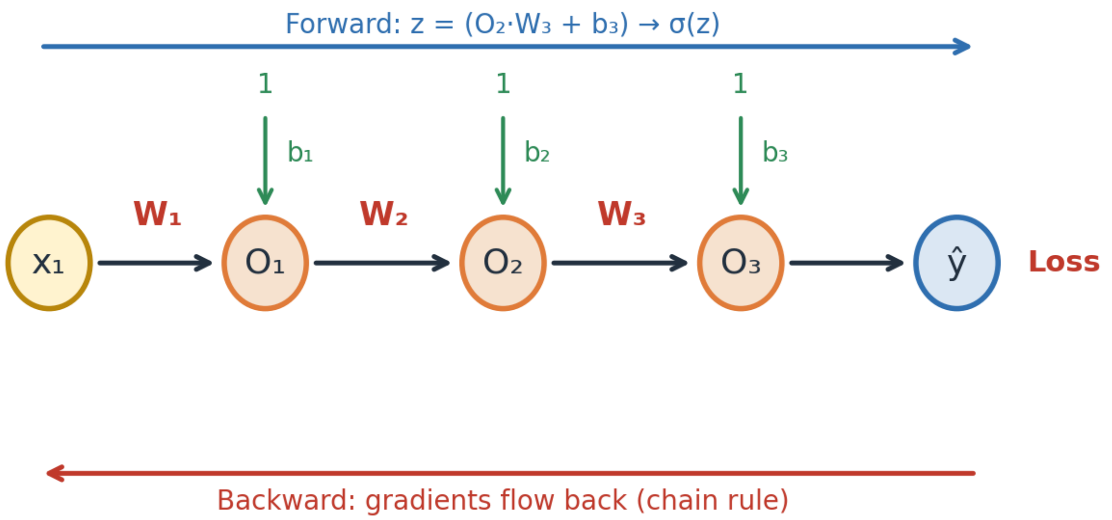


Figure 7.1 — A chain of neurons. Forward pass computes $z = (O_2 \cdot W_3 + b_3) \rightarrow \sigma(z)$; backward pass sends gradients back.

The forward step at the last neuron is simply:

$$z = (O_2 \cdot W_3 + b_3) \;\rightarrow\; \sigma(z)$$

7.3 Why gradients explode — following the chain rule

To update an early weight, backpropagation **multiplies** the effects along the whole path using the chain rule of derivatives:

$$\frac{\partial L}{\partial W_{old}} = \frac{\partial L}{\partial O_3} \cdot \frac{\partial O_3}{\partial O_2} \cdot \frac{\partial O_2}{\partial O_1} \cdot \frac{\partial O_1}{\partial W_{old}}$$

The danger is in the multiplication. If **each** term in that product happens to be a **big** number, then big × big × big × big compounds into an enormous value:

$$\text{big} \times \text{big} \times \text{big} \times \text{big} = \text{Very high value}$$

When that huge gradient enters the update rule, the weight takes a giant, uncontrolled jump — that is the **exploding gradient**. Now, where do these "big" terms come from?

Let us zoom into one of the middle terms and expand it, exactly as in the notes:

$$\frac{\partial O_3}{\partial O_2} = \frac{\partial \sigma(z)}{\partial z} \cdot \frac{\partial z}{\partial O_2}$$

$$= [\, 0 - 0.25 \,] \times \frac{\partial (O_2 W_3 + b_3)}{\partial O_2} = [\, 0 - 0.25 \,] \times W_3$$

Notice the two pieces. The first, **∂σ(z)/∂z**, is the sigmoid derivative — always a small value between **0 and 0.25**. The second piece turns out to be simply the weight **W_3**. So the whole term equals a small number multiplied by W_3.

The key insight

The sigmoid derivative is small (≤ 0.25), so on its own it would cause *vanishing*. But if the **weight is initialized very large** — say W_3 = 500 to 1000 — then even 0.25 × W_3 becomes a big number. Multiply several such big numbers through the chain and the gradient **explodes**.

$$[\, 0 - 0.25 \,] \times W_3 \;\; (W_3 = 500 \text{ to } 1000) \;\Rightarrow\; \text{large}$$

Conclusion: the root cause of the exploding gradient is **poorly chosen (too large) initial weights**. This gives us a direct cure — initialize the weights sensibly in the first place.

7.4 Vanishing vs Exploding — two sides of the same coin

Both problems come from the **same** chain-rule multiplication during backpropagation.

The difference is only the size of the terms being multiplied:

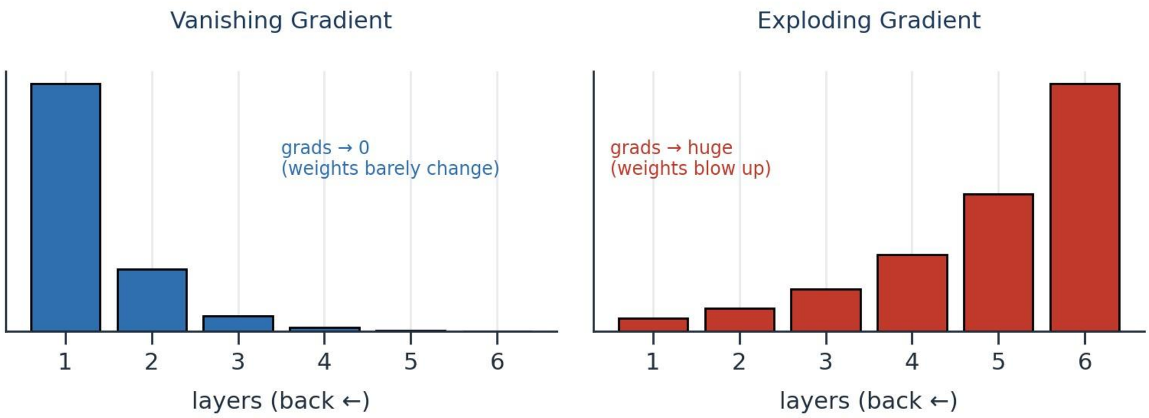


Figure 7.2 — Multiply small numbers → gradient vanishes. Multiply large numbers → gradient explodes.

	Vanishing Gradient	**Exploding Gradient**
What happens	Gradients shrink toward 0	Gradients grow toward huge values
Effect on weights	W_new ≈ W_old (no learning)	W_new ≫ W_old (unstable jumps)
Common cause	Sigmoid/Tanh (small derivatives)	Weights initialized too large
Symptom	Training stalls, no improvement	Loss becomes NaN / wild swings

The good news is that a single, well-chosen strategy — **smart weight initialization** — helps prevent *both* extremes by keeping the starting gradients in a healthy range.

7.5 Three golden rules for initializing weights

Before looking at the formulas, remember what "good" weights look like. The notes highlight three key points:

- **Weights should be small.** Large weights are exactly what triggers exploding gradients.
- **Weights should not all be the same.** If every weight is identical, all neurons learn the same thing and the network wastes its capacity (this is called *symmetry*).
- **Weights should have good variance.** A healthy spread of values lets different neurons capture different patterns.

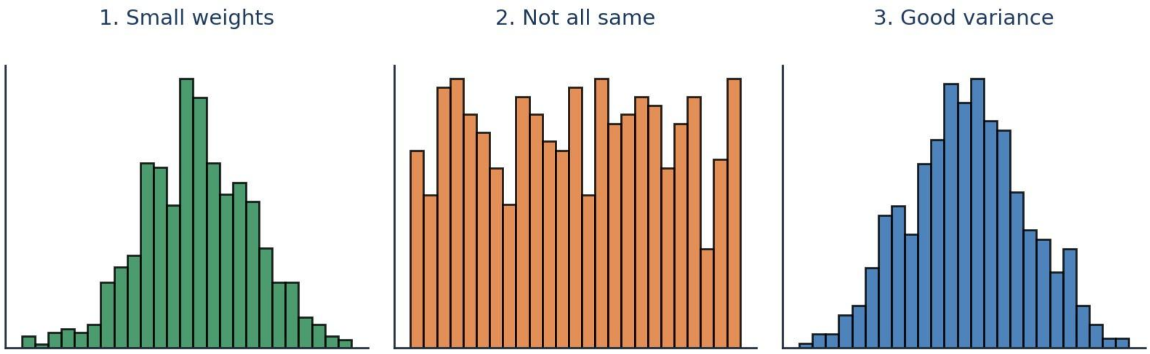


Figure 7.3 — Good initial weights are small, varied (not identical), and have a sensible spread (variance).

Why "not the same" matters

If all weights start equal, every neuron receives the same gradient and updates identically forever — they stay clones of each other. Random initialization **breaks this symmetry** so neurons can specialise.

7.6 Weight Initialization Techniques

The techniques below all follow the three golden rules. They differ mainly in **how** they set the spread, using the number of **inputs** (fan-in) and sometimes **outputs** (fan-out) of a layer. The little network below (input = 3, output = 3) is the reference used in the notes.

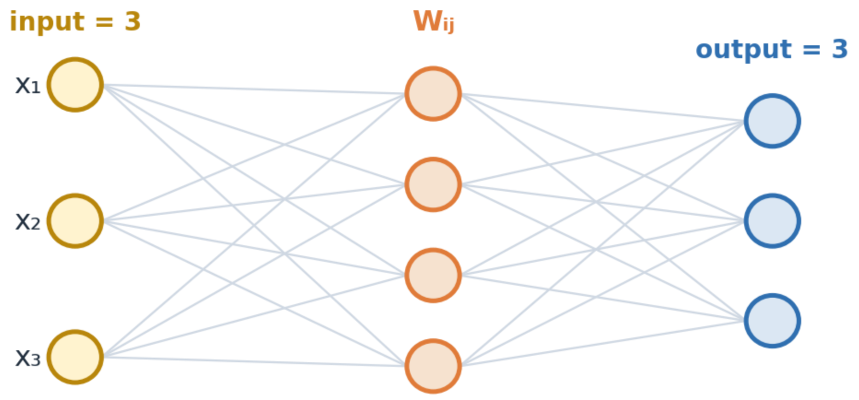


Figure 7.4 — 'input' = number of incoming connections; 'output' = number of outgoing connections for a layer.

1) Uniform Distribution

The simplest method. Each weight is drawn randomly from a uniform range that shrinks as the number of inputs grows — so more inputs automatically means smaller weights:

$$W_{ij} \sim U\left[\frac{-1}{\sqrt{\text{input}}}, \frac{1}{\sqrt{\text{input}}}\right]$$

For example, with 3 inputs the range becomes:

$$\text{input} = 3 \;\Rightarrow\; W_{ij} \sim U\left[\frac{-1}{\sqrt{3}}, \frac{1}{\sqrt{3}}\right]$$

2) Xavier / Glorot Initialization

Proposed by researcher **Xavier Glorot**, this method uses **both** the input and output counts, which makes it work very well with **Sigmoid** and **Tanh** activations. It comes in two forms:

Xavier Normal — draw from a normal (bell-curve) distribution:

$$W_{ij} \sim N(0, \sigma), \quad \sigma = \sqrt{\frac{2}{\text{input} + \text{output}}}$$

Xavier Uniform— draw from a uniform range:

$$W_{ij} \sim U\left[\frac{-\sqrt{6}}{\sqrt{\text{input} + \text{output}}}, \frac{\sqrt{6}}{\sqrt{\text{input} + \text{output}}}\right]$$

3) Kaiming He Initialization

This method uses only the **input** count and is specially designed for **ReLU** and its variants — the activations we now use most in hidden layers (Part 4). It too has a normal and a uniform form:

He Normal — normal distribution:

$$W_{ij} \sim N(0, \sigma), \quad \sigma = \sqrt{\frac{2}{\text{input}}}$$

He Uniform — uniform range:

$$W_{ij} \sim U\left[\frac{-\sqrt{6}}{\sqrt{\text{input}}}, \frac{\sqrt{6}}{\sqrt{\text{input}}}\right]$$

7.7 Which technique should I use?

Technique	Uses	Best paired with	Distribution forms
Uniform	input (fan-in)	Simple / small networks	Uniform
Xavier / Glorot	input + output	Sigmoid, Tanh	Normal & Uniform
Kaiming He	input (fan-in)	ReLU, Leaky ReLU, ELU	Normal & Uniform

✓ Advantages

- Prevents exploding AND vanishinggradients
- Keeps weights small with good variance
- Breaks neuron symmetry (weights notidentical)
- Speeds up and stabilises training

✗ Disadvantages

- The right choice depends on the activationfunction
- Not a complete cure alone — oftencombined with gradient clipping, batch norm, etc.

Part 7 in one line

Exploding gradients happen when the chain-rule product of large terms blows up, and the usual culprit is **weights that start too big**. The fix is smart **weight initialization** — keep weights small, varied, and not identical — using **Uniform**, **Xavier/Glorot** (for Sigmoid/Tanh), or **Kaiming He** (for ReLU). This tames both the exploding and vanishing gradient problems.

— End of Part 7 —

PART 8

The Dropout Layer

In Part 7 we tamed exploding gradients with good weight initialization. Now we meet a different enemy — overfitting — and a wonderfully simple trick that fights it by randomly switching neurons off during training.

8.1 The problem: overfitting

Imagine a student who memorises every answer in a practice book instead of understanding the subject. In the practice test they score brilliantly, but in the real exam — with new questions — they struggle. A neural network can behave the same way. When it **memorises the training data** instead of learning the general pattern, we call it **overfitting**.

The tell-tale sign is a big gap between training and test performance, exactly as noted:

$$\text{Train Acc} = 90\% \quad \gg \quad \text{Test Acc} = 60\%$$

A model that scores **90% on training** but only **60% on the test** has clearly memorised rather than generalised. What we actually want is a **generalized model** — one that performs well on data it has never seen before.

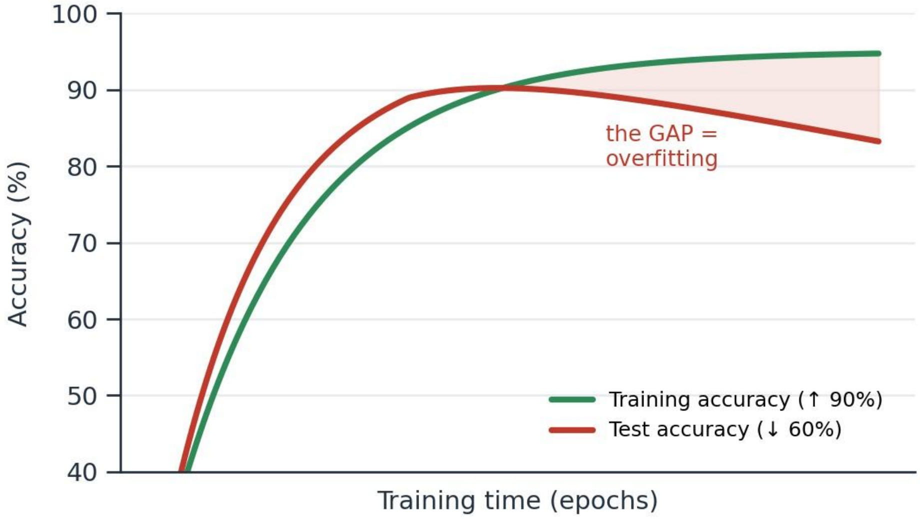


Figure 8.1 — Overfitting: training accuracy keeps climbing while test accuracy stalls or drops. The gap is the problem.

Why does overfitting happen?

A deep network has a huge number of neurons and weights. With so much capacity, it can create an overly complex mapping that fits every tiny quirk (and even the noise) in the training data. Powerful models are the *most* prone to this.

8.2 A lesson from Random Forest

This exact problem already has a famous solution in classical machine learning. A single **Decision Tree** tends to overfit — it grows deep and memorises the data. The **Random Forest** fixes this by building **many** trees, where each tree only sees a **random subset** of the rows (row sampling) and a random subset of the features (feature sampling). No single tree sees everything, so no single tree can memorise everything — and combining them gives a **generalized model**.

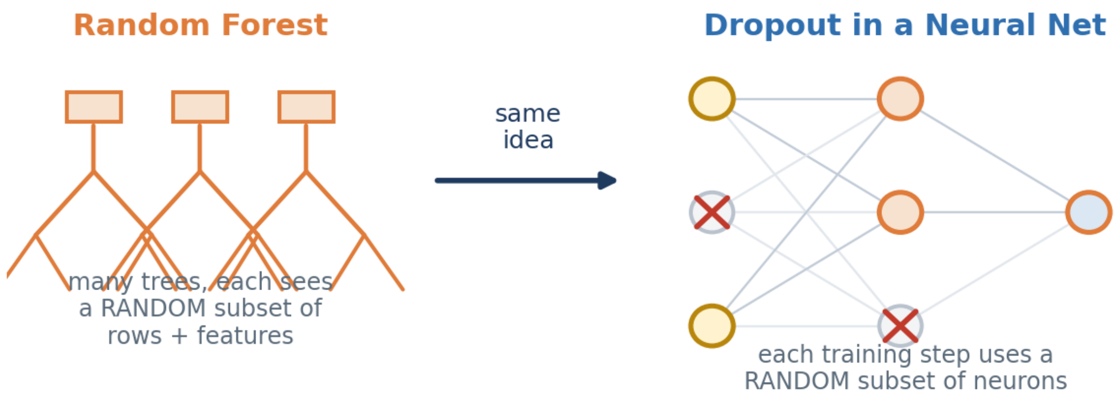


Figure 8.2 — Random Forest generalizes by giving each tree a random slice of the data. Dropout applies the same idea to neurons.

Dropout brings this same idea into neural networks. Instead of using every neuron on every training step, we randomly switch some of them off — so the network is forced to learn robust patterns rather than relying on any single neuron.

8.3 What is the Dropout Layer?

Dropout is a regularization technique. On each training step, it randomly **deactivates (drops)** a fraction of the neurons in a layer — they output nothing, as if they were temporarily removed from the network. Every step drops a *different* random set, so the network effectively trains many slightly different "thinner" networks and averages their strengths.

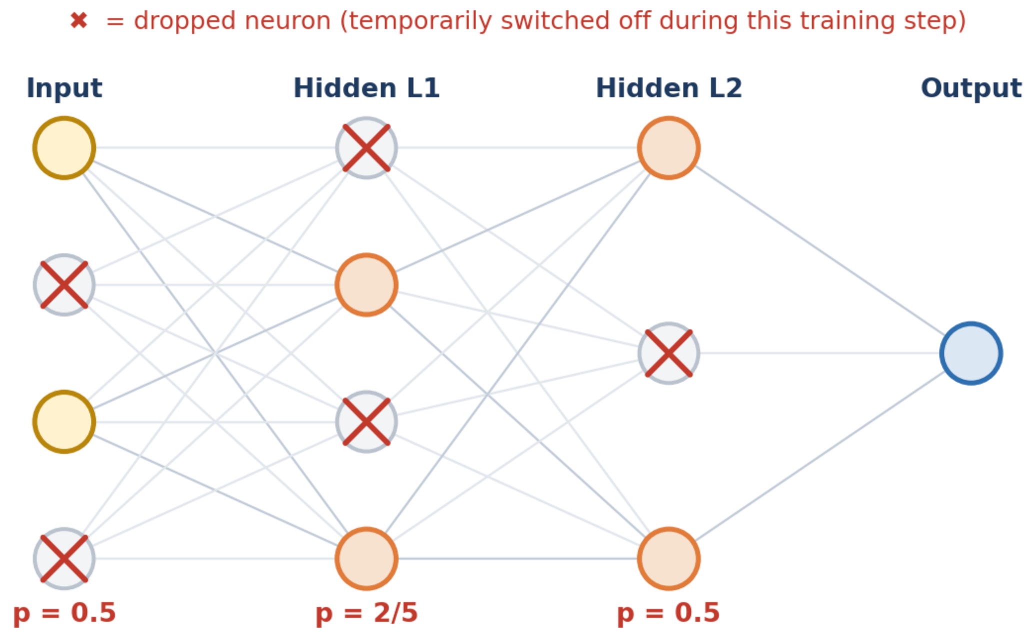


Figure 8.3 — On this training step the crossed-out neurons are switched off; the remaining ones must still do the job.

The key intuition

Because any neuron might vanish at any moment, no neuron can become a “star performer” that the others lean on. Every neuron must pull its weight. This **breaks over-reliance** (called co-adaptation) and forces the network to spread out what it has learned — giving a more robust, generalized model.

8.4 The dropout ratio p (a hyperparameter)

How many neurons should we keep? That is controlled by **p**, the dropout ratio. It is a **hyperparameter** — a value *you* choose before training, not something the network learns. It always lies between 0 and 1:

$$0 \leq p \leq 1$$

In these notes, **p is the fraction of neurons kept active** in a layer. Different layers can use different values of p — in Figure 8.3, the input layer uses p = 0.5, the first hidden layer uses p = 2/5, and so on. Choosing p is a balancing act:

Value of p	What it means	Effect
p close to 1	Keep almost all neurons	Very little regularization; may still overfit
p around 0.5	Keep about half	A common, balanced choice for hidden layers
p close to 0	Drop almost all neurons	Too aggressive; network can't learn (underfitting)

8.5 Training time vs Test time — an important difference

Here is the subtle part that trips up beginners. Dropout is only active during **training**. At **test time** (when we actually use the model), we want a stable, deterministic answer, so we do **not** drop any neurons — every neuron stays on.

But that creates a mismatch. During training each neuron only received signals from a *fraction* (p) of the neurons before it; now at test time it suddenly receives signals from *all* of them, so the incoming values would be too large. To keep things balanced, we **scale every weight by p** at test time:

$$W_{test} = W_{trained} \times p$$

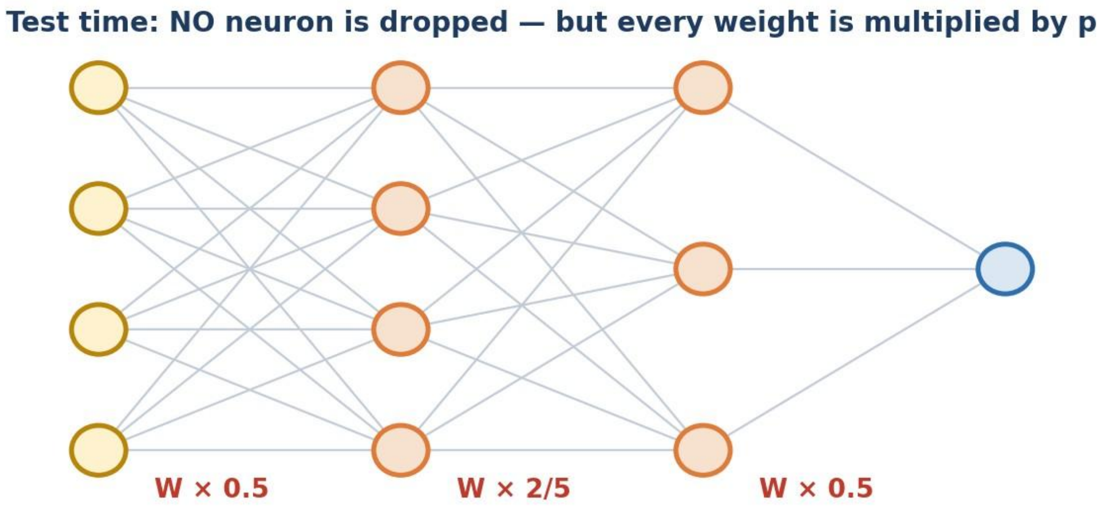


Figure 8.4 — At test time all neurons are active, but each weight is multiplied by p (e.g. W×0.5) to match the training scale.

One-line rule

Training: randomly drop neurons (keep a fraction p). **Testing:** keep all neurons, but multiply their weights by p. This keeps the total signal consistent between the two phases.

8.6 Advantages and disadvantages

✓ Advantages

- Reduces overfitting effectively
- Forces neurons to be independent androbust (no co-adaptation)
- Acts like training many networks at once — similar to a Random Forest • Very simple and cheap to add

✗ Disadvantages

- Training takes longer to converge (thenetwork changes each step)
- p is another hyperparameter you musttune
- Too high a drop rate causes underfitting
- Needs the weight-scaling step at test time

Part 8 in one line

Overfitting is when a network memorises the training data (high train accuracy, low test accuracy). **Dropout** fights it by randomly switching off a fraction of neurons (kept ratio **p**, with $0 \le p \le 1$) during training — the same generalization trick a Random Forest uses — and then scaling weights by p at test time. The result is a robust, **generalized model**.

— End of Part 8 —

TensorFlow — Building an ANN in Practice

So far (Parts 1–8) we learned how a neural network works on the inside. Now we put it to work. This part shows what TensorFlow is and walks through a complete, hands-on ANN — including data preprocessing — using the same Pass/Fail student example from your notes.

9.1 What is TensorFlow?

TensorFlow is a free, open-source machine-learning framework created by **Google** (released in 2015). It handles the entire deep-learning workflow — preparing data, building a model, training it, and using it — and can run on laptops, servers, phones, and browsers. The name has two parts:

- **Tensor** = a multi-dimensional array (how it stores numbers — a single value, a list, a table, or more).
- **Flow** = the computations are arranged as a graph through which the tensors *flow*, step by step.

The single most useful thing it does for us: **automatic differentiation**. It computes all the gradients (∂L/∂W) automatically, which means TensorFlow runs the entire **backpropagation** and **gradient descent** from Parts 3, 6 and 7 *for you* — you never hand-code a single derivative.

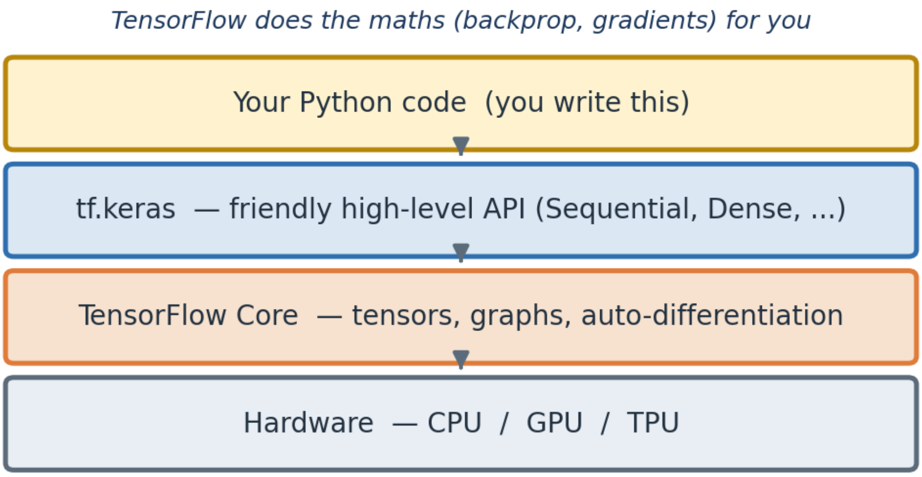


Figure 9.1 — You write code with tf.keras (the friendly API); TensorFlow Core does the heavy maths on CPU/GPU/TPU.

Keras vs TensorFlow — clearing the confusion
Keras is not a competitor to TensorFlow; it lives *inside* it as **tf.keras**, the official high-level API. When you write **from tensorflow.keras…** you are using Keras as the easy front-door to TensorFlow's powerful engine. That is exactly what we do below.

9.2 The six-step workflow

Almost every ANN project in TensorFlow follows the same six steps. Keep this map in mind — the rest of this part simply walks through each box.

Figure 9.2 — The standard TensorFlow/Keras workflow, from raw data to a saved, prediction-ready model.

9.3 Step 1 — Data preprocessing

A network can only learn from **clean numbers**. Real data is messy, so before any training we prepare it. We will predict whether a student **Passes or Fails** from four features: **IQ**, **StudyHours**, **PlayHours**, and **School** (a text category). Preprocessing turns this into tidy numeric arrays.

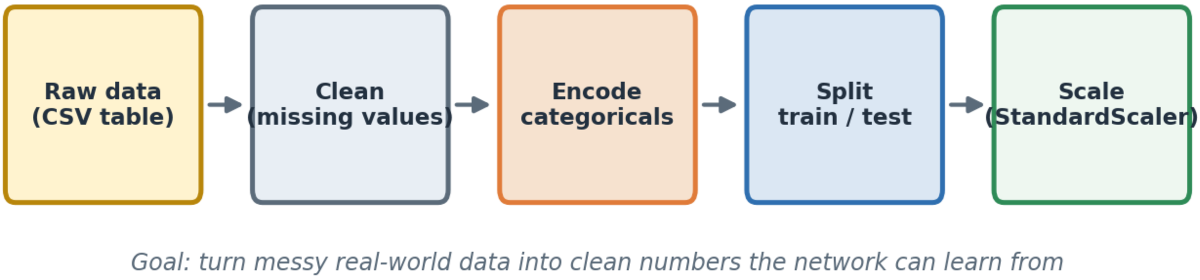


Figure 9.3 — The preprocessing pipeline: clean → encode → split → scale.

Why each step matters:

- **Encode categoricals.** A network cannot read the word “Private”, so we turn the **School** column into 0/1 columns (one-hot encoding).
- **Train/test split.** We keep some data hidden for testing, so we can measure *generalization* — the very idea behind overfitting and dropout in Part 8.
- **Feature scaling.** IQ (~80–140) and StudyHours (~0–10) live on very different scales. Scaling puts them on a level field, which keeps gradients healthy (Part 7) and helps the network learn faster.

Golden rule of scaling (validated)

Always **fit the scaler on the training data only**, then apply it to the test data. Fitting on test data would leak information and give falsely high scores. This pipeline was run and verified end-to-end before printing here.

● **Step 1 — Preprocessing (pandas + scikit-learn)**

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1a. Load the data (a CSV table of students)
df = pd.read_csv("students.csv")
# Columns: IQ, StudyHours, PlayHours, School, Result
# 1b. Encode the text column 'School' into 0/1 columns
df = pd.get_dummies(
    df,
    columns=["School"],
    drop_first=True
)

# 1c. Separate inputs (X) from the target label (y)
X = df.drop("Result", axis=1).values      # Features
y = df["Result"].values                   # Target (1 = Pass, 0 = Fail)

# 1d. Split into training and test sets (80% / 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
# 1e. Scale features
# Fit only on the training data, then transform both datasets
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

‣ Line-by-line: what each function does, and why

Reading the preprocessing code from top to bottom

`import numpy as np, pandas as pd`	**What:** loads two toolkits — NumPy (fast number arrays) and pandas (spreadsheet-like tables). **Why:** our data starts as a table, so we need pandas to read it and NumPy to hand clean number arrays to the network. **How:** *import ... as* gives each library a short nickname (np, pd) so we type less.
`train_test_split`	**What:** a scikit-learn function that splits data into a training part and a testing part. **Why:** we must test the model on data it has never seen, to check real learning (the generalization idea from Part 8). **How:** imported here, actually called in step 1d.

`StandardScaler`	**What:** a scikit-learn tool that rescales each feature to have mean 0 and spread 1. **Why:** IQ (~80–140) and StudyHours (~0–10) are on very different scales; leaving them unequal makes training unstable. **How:** imported now, used in step 1e.
`pd.read_csv("students.csv")`	**What:** opens the CSV file and loads it into a pandas table called *df* (short for dataframe). **Why:** it is the easiest way to bring a real dataset into Python. **How:** pandas reads the comma-separated rows and columns and builds a table you can slice and edit.
`pd.get_dummies( df, columns=["School"], drop_first=True)`	**What:** converts the text column *School* into 0/1 number columns (one-hot encoding). **Why:** a network cannot read the word 'Private' — only numbers. **How:** it makes one column per category; *drop_first=True* removes one redundant column to avoid duplication.
`df.drop("Result", axis=1) .values`	**What:** takes every column *except* the answer column and converts it to a plain NumPy array *X* (the inputs). **Why:** the model needs features and the answer kept separate. **How:** *drop* removes the column, *axis=1* means 'a column', and *.values* turns the table into a number array.
`df["Result"].values`	**What:** pulls out just the answer column into *y* (1 = Pass, 0 = Fail). **Why:** this is the truth the network tries to predict. **How:** square brackets select the column; *.values* makes it an array.
`train_test_split( X, y, test_size=0.2, random_state=42, stratify=y)`	**What:** randomly divides the data — 80% to train, 20% to test. **Why:** a held-out test set is the honest exam. **How:** *test_size=0.2* keeps 20% for testing; *random_state=42* makes the split repeatable; *stratify=y* keeps the Pass/Fail ratio balanced in both parts.
`StandardScaler()`	**What:** creates the scaler object. **Why:** we need one scaler that remembers the training statistics. **How:** stores the mean and spread it will learn from the training data.
`scaler.fit_transform( X_train)`	**What:** two actions at once — *fit* learns the mean/spread from the training data, then *transform* rescales it. **Why:** the network learns faster and gradients stay healthy (Part 7). **How:** subtracts the mean and divides by the spread, feature by feature.
`scaler.transform( X_test)`	**What:** rescales the test data using the *training* statistics only. **Why:** using test data to fit the scaler would leak information and fake a high score. **How:** note there is no *fit* here on purpose — it reuses what was learned from the training set.

Golden rule (validated)

Always **fit the scaler on training data only**, then transform both sets. This whole pipeline was run and verified end-to-end before this document was produced (correct shapes, 5 input features, no leakage).

9.4 Steps 2–3 — Build and compile the ANN

Now the exciting part: every line of the model below is a topic from your notes. Each **Dense** layer is a full layer of the perceptrons from Part 2, computing exactly the weighted-sum-plus-activation you already know:

$$\text{output} = \text{activation}\,(\,X \cdot W + b\,)$$

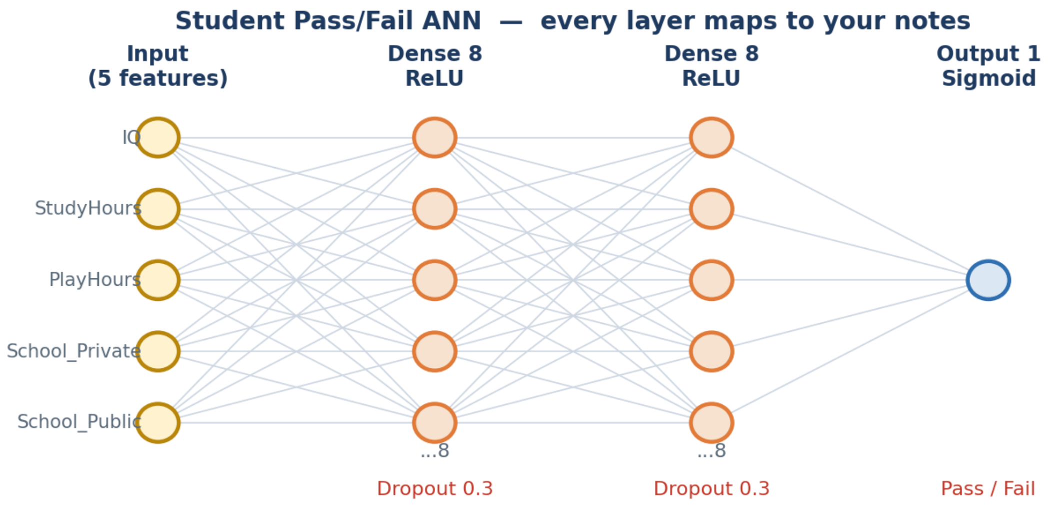


Figure 9.4 — Our ANN: 5 inputs → Dense(8, ReLU) → Dense(8, ReLU) → Dense(1, Sigmoid), with Dropout in between.

• Steps 2–3 — Build the model, then compile it

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
# Number of input features
n_features = X_train.shape[1]      # = 5 after encoding
# STEP 2 - Build the neural network architecture
model = Sequential([
    # Hidden Layer 1
    Dense(
        8,
        activation="relu",                 # ReLU activation
        kernel_initializer="he_normal",    # He initialization for ReLU
        input_shape=(n_features,)
    ),
    # Dropout to reduce overfitting
    Dropout(0.3),
    # Hidden Layer 2
    Dense(8,activation="relu",kernel_initializer="he_normal"),
    # Dropout to reduce overfitting
    Dropout(0.3),
    # Output Layer
    Dense(1, activation="sigmoid")               # Outputs probability (0 to 1)
])
```

```
# STEP 3 - Compile the model
model.compile(
    optimizer="adam",                    # Adam optimizer
    loss="binary_crossentropy",          # Loss for binary classification
    metrics=["accuracy"]                 # Evaluation metric
)
# Display model architecture
model.summary()
```

Line-by-line: what each function does, and why

Reading the model-building code from top to bottom

`import tensorflow as tf`	**What:** loads the TensorFlow library. **Why:** it is the engine that will build, train and run our network. **How:** nicknamed *tf* so we can call its tools quickly.
`Sequential`	**What:** a model type where layers are stacked in a straight line, one after another. **Why:** our ANN is a simple front-to-back stack, which is exactly what Sequential is for. **How:** imported from tensorflow.keras.models.
`Dense, Dropout`	**What:** two layer types — Dense (a fully-connected layer of neurons) and Dropout (randomly switches neurons off). **Why:** Dense does the learning; Dropout fights overfitting (Part 8). **How:** imported from tensorflow.keras.layers.
`X_train.shape[1]`	**What:** reads how many columns (features) the input has — here 5. **Why:** the first layer must know how many inputs to expect. **How:** *.shape* gives (rows, columns); index [1] picks the columns.
`Sequential([ ... ])`	**What:** builds the network by listing its layers in order. **Why:** this defines the whole architecture in one place. **How:** the list runs top-to-bottom, so data flows input → first layer → ... → output.
`Dense(8, activation='relu')`	**What:** a hidden layer of 8 neurons using the ReLU activation. **Why:** 8 neurons give it capacity to learn patterns; ReLU is fast and avoids the vanishing-gradient problem (Part 4). **How:** each neuron does output = ReLU(X·W + b).
`kernel_initializer= 'he_normal'`	**What:** sets the starting weights using He initialization. **Why:** good starting weights prevent exploding/vanishing gradients (Part 7); He is the right choice for ReLU layers. **How:** draws small random weights scaled by the number of inputs.
`input_shape= (n_features,)`	**What:** tells the first layer how many inputs arrive (5). **Why:** only the first layer needs this; the rest infer it automatically. **How:** a tuple describing the shape of one input row.

`Dropout(0.3)`	**What:** during training, randomly switches off 30% of this layer's neurons each step. **Why:** it stops the network memorising and improves generalization (Part 8). **How:** at test time it is automatically turned off; only active while learning.
`Dense(1, activation='sigmoid')`	**What:** the output layer — a single neuron with sigmoid. **Why:** Pass/Fail is one yes/no answer, and sigmoid gives a probability between 0 and 1 (Parts 2 & 4). **How:** outputs 1/(1+e^−z).
`model.compile(...)`	**What:** gets the model ready to train by fixing three choices. **Why:** the network needs to know how to measure error and how to improve. **How:** takes optimizer, loss, and metrics (next three rows).
`optimizer='adam'`	**What:** the rule that updates the weights. **Why:** Adam is the reliable default — it combines momentum with an adaptive learning rate (Part 6). **How:** after each batch it nudges every weight to lower the loss.
`loss= 'binary_crossentropy'`	**What:** the score of how wrong the prediction is. **Why:** it is the correct loss for a 2-class (Pass/Fail) problem (Part 5). **How:** it is small when the predicted probability matches the true label, large when confidently wrong.
`metrics=['accuracy']`	**What:** an extra, human-friendly score shown during training. **Why:** accuracy (% correct) is easier to read than raw loss. **How:** it is only reported, not used to train.
`model.summary()`	**What:** prints a table of the layers and how many weights the model has. **Why:** a quick check that the architecture is what you intended. **How:** lists each layer's output shape and parameter count.

9.5 Steps 4–6 — Train, evaluate, and predict

With the model compiled, **training** is a single call. Behind that one line, TensorFlow performs the full loop from your notes — forward propagation (Part 3) → loss (Part 5) → backpropagation with the chain rule (Part 3) → weight update via Adam (Part 6) — repeated for every batch and every epoch.

● Steps 4–6 — Train, evaluate, predict, save

```python
# STEP 4 - Train the model
# One epoch = one complete pass over the training dataset
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,          # Mini-batch size
    validation_split=0.1    # Reserve 10% of training data for validation
)
# STEP 5 - Evaluate the model on the unseen test set
loss, acc = model.evaluate(X_test, y_test)
print(f"Test accuracy: {acc:.3f}")
# STEP 6 - Predict for a new student
# Apply the same preprocessing (feature scaling) used during training
new_student = scaler.transform([
    [120, 7, 2, 1, 0]
])
# Predict the probability of passing
prob = model.predict(new_student)[0][0]
# Convert probability to class label
print("Pass" if prob > 0.5 else "Fail", f"(p={prob:.2f})")
# Save the trained model for future use
model.save("student_ann.h5")
```

▸ **Line-by-line: what each function does, and why**

Reading the training / prediction code from top to bottom

`model.fit( X_train, y_train)`	**What:** trains the network on the training data. **Why:** this is where actual learning happens — the weights improve step by step. **How:** it repeatedly does forward pass → loss → backprop → weight update, all handled by TensorFlow.
`epochs=50`	**What:** how many complete passes over the whole dataset. **Why:** one pass is rarely enough; repeating lets the network keep improving. **How:** 50 means the data is shown to the model 50 times.
`batch_size=32`	**What:** update the weights after every 32 samples, not after all data. **Why:** this is mini-batch gradient descent — a good balance of speed and stability (Part 6). **How:** data is chopped into groups of 32 for each small update.
`validation_split=0.1`	**What:** quietly keeps 10% of the training data aside to check progress each epoch. **Why:** it warns you if the model starts overfitting (Part 8). **How:** after each epoch it reports accuracy on this held-back slice.

`history = ...`	**What:** stores the accuracy/loss values recorded during training. **Why:** so you can plot learning curves later. **How:** fit() returns a record you save in the variable *history*.
`model.evaluate( X_test, y_test)`	**What:** measures the trained model on the unseen test set. **Why:** this is the honest exam — the true test of generalization. **How:** it runs a forward pass over the test data and returns the loss and accuracy.
`print(f"...{acc:.3f}")`	**What:** shows the test accuracy on screen. **Why:** to read the final result. **How:** an f-string inserts the value; *:.3f* rounds it to 3 decimals.
`scaler.transform( [[120,7,2,1,0]])`	**What:** scales one new student's data the *same* way as training. **Why:** the model only understands scaled inputs, so new data must get identical treatment. **How:** reuses the fitted scaler — no re-fitting.
`model.predict( new_student)[0][0]`	**What:** runs a pure forward pass and returns the Pass probability. **Why:** this is the model actually being used to make a decision (Part 3). **How:** the double [0][0] pulls the single number out of the result array.
`"Pass" if prob>0.5 else "Fail"`	**What:** turns the probability into a clear Yes/No. **Why:** humans want a decision, not a decimal. **How:** the 0.5 threshold is the same rule a single perceptron used in Part 2.
`model.save( "student_ann.h5")`	**What:** saves the trained model to a file. **Why:** so you can reuse it later without training again. **How:** stores the architecture and all learned weights in one .h5 file you can reload with load_model().

The output of the final Dense neuron is a probability between 0 and 1 (thanks to sigmoid). We simply read it as **Pass** if it is above 0.5, otherwise **Fail** — the very same decision a single perceptron made in Part 2, now backed by a trained multi-layer network.

Loss used here (Part 5 reminder)

For this two-class problem the loss is **Binary Cross-Entropy**. It is small when the predicted probability is close to the true label and large when it is confidently wrong:

$$\text{Loss} = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$$

9.6 How every line connects to your notes

This is the heart of the request — seeing your whole journey (Parts 1–8) appear inside a few lines of real code. Keep this table beside you when you code:

Code you write	Notes concept	Part
Dense(8, ...)	A layer of perceptrons: output = act(X·W + b)	Part 2, 3
activation='relu'	ReLU activation (fast, avoids vanishing gradient)	Part 4
activation='sigmoid'	Sigmoid squashes output to a 0–1 probability	Part 2, 4
kernel_initializer='he_normal'	He weight initialization (for ReLU layers)	Part 7
Dropout(0.3)	Randomly switch off neurons to fight overfitting	Part 8
loss='binary_crossentropy'	Loss for 2-class classification	Part 5
optimizer='adam'	Adam = Momentum + adaptive learning rate	Part 6
batch_size=32	Mini-batch gradient descent	Part 6
model.fit(...)	Forward prop + backprop + weight update loop	Part 3, 6
model.predict(...)	A pure forward-propagation pass	Part 3
StandardScaler / encoding	Clean numeric inputs; helps healthy gradients	Part 7

Part 9 in one line

TensorFlow (via **tf.keras**) lets you build a working ANN in a handful of lines: preprocess the data into clean numbers, stack **Dense** layers with the activations, initializers and dropout from your notes, **compile** with a loss + optimizer, then **fit** — and TensorFlow automatically runs the forward pass, backpropagation, and weight updates you learned across Parts 1–8.

Note on validation

The preprocessing pipeline and the network's forward-pass maths in this part were executed and verified before this document was produced: encoding → split → scaling gave correct shapes (5 input features), and the model output stayed a valid probability between 0 and 1 with a finite loss.

— End of Part 9 —